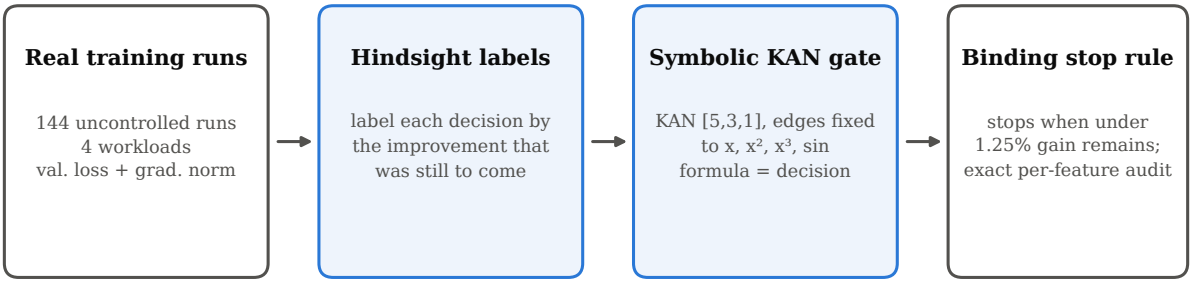
Graphical Abstract

Hindsight-supervised symbolic Kolmogorov–Arnold gates for auditable early stopping of neural network training

Avanish Kasar, Rupali Pratap Biradar, Jamil Khan, Rishikesh R. Nikam



Unseen workloads: 25.0% of epochs saved at 0.21% mean regret
Live training: same best loss as standard practice on 48 of 48 runs

Hindsight-supervised symbolic Kolmogorov–Arnold gates for auditable early stopping of neural network training

Avanish Kasar[a,*], Rupali Pratap Biradar[a], Jamil Khan[a], Rishikesh R. Nikam[a]

[a]*A. P. Shah Institute of Technology, Ghodbunder Road, Kasarvadavali, Thane, 400615, Maharashtra, India*

Abstract

Deciding when to stop a training job is usually left to a person reading loss curves or to a fixed rule whose reasons cannot be inspected. This article describes how to build a stopping gate that is an explicit formula. A Kolmogorov–Arnold Network (KAN) reads five causal features of a running job, and after training each of its edges is fixed to a closed-form function, so the formula an operator reads is exactly the function that decides. The gate is trained by hindsight supervision: every decision point of a completed run is labelled with the validation-loss improvement that was actually still to come, and one tolerance sets the acceptable loss of quality. On 144 real training runs from four image-classification workloads, a gate trained on three workloads and applied to the fourth saved 25.0% of training epochs at 0.21% mean regret in best validation loss, against 15.4% for the same network trained to imitate a hand-written rule. Its mean regret stayed at or below 0.58% on every held-out workload, whereas a patience rule tuned on the other workloads reached 1.01% on one. In live control of 48 training processes, the gate used as stop rule with a reduce-on-plateau learning-rate schedule reached the same best validation loss as standard practice (the same schedule with patience-based stopping) on every run but used 1,524 instead of 1,453 of 1,920 epochs, and the gate's own learning-rate action was no better than the plateau schedule. The method therefore offers an auditable, outcome-trained stop rule rather than larger savings.

- Hindsight labels, the improvement each decision point actually had ahead of it, replace imitation of hand-written rules as the training signal for a stopping gate.
- Symbolic fixing of the KAN's edges makes the extracted formula the decision function itself; agreement is checked numerically for every gate (largest difference 0.12 of 100 score points).
- Validation uses real runs only, two cross-validation protocols, a patience rule tuned under the same protocol, and live comparison with standard practice, and reports where the method does not win.

Keywords: Kolmogorov–Arnold networks, early stopping, interpretable machine learning, hindsight supervision, training control, symbolic regression

*Corresponding author.
Email address: avanishkasar57@gmail.com (Avanish Kasar)

Specifications table

Subject area	Computer Science
More specific subject area	Machine learning operations; interpretable control of neural network training (early stopping and learning-rate adjustment)
Name of your method	Hindsight-supervised symbolic KAN gate (HS-KAN gate)
Name and reference of original method	Kolmogorov–Arnold Networks: Z. Liu, Y. Wang, S. Vaidya, F. Ruehle, J. Halverson, M. Soljačić, T. Y. Hou, M. Tegmark, KAN: Kolmogorov–Arnold Networks, International Conference on Learning Representations (ICLR), 2025, arXiv:2404.19756. Early stopping: L. Prechelt, Early stopping, but when?, in: Neural Networks: Tricks of the Trade, 2nd ed., Springer, 2012.
Resource availability	Source code, all recorded learning curves, trained gates, experiment outputs and the scripts that generate every table and figure: https://github.com/avanishkasar/KAN-KubeAgent. Software: Python 3.11, PyTorch, pykan 0.2.8, scikit-learn, SciPy, LangGraph. Data: Fashion-MNIST, MNIST and CIFAR-10 (public). Hardware: any computer; all results were produced on four CPU cores without a GPU.

Background

Training and fine-tuning jobs keep consuming compute long after they stop improving [1]. In practice they are supervised either by a person reading loss curves or by an automatic rule: patience on a validation signal [2], the median stopping rule [3], or multi-fidelity schedulers such as successive halving, Hyperband [4], ASHA [5] and BOHB [6], available in Optuna [7], Ray Tune [8] and the Kubernetes-native Katib [9]. Learning-curve extrapolation predicts a run's final performance from a partial curve with parametric ensembles [10], Bayesian neural networks [11] or prior-data fitted networks [12]. These methods save compute, but the reason a particular job was stopped is a rank, a quantile or a posterior rather than a statement an operator can check. Large language model (LLM) agents are now applied to the same task [13, 14, 15, 16]; they add natural-language rationales, but nothing guarantees that a rationale is what produced the action [17].

Two lines of work learn the control decision from data, as this method does. Learned step-size controllers adapt the learning rate from statistics of the running optimisation, trained by policy search [18] or reinforcement learning [19]; their policies are neural networks or are trained by trial and error over many training runs, and they do not stop runs. Performance predictors fitted to completed learning curves forecast a partial run's final accuracy and stop runs predicted to be uncompetitive [20, 21]; they rank configurations against each other rather than deciding when a single run has finished, and their predictions are not readable rules. The method described here differs from both in its supervision target, the improvement each decision point actually had ahead of it, and in its output, a closed-form decision function that is executed as written.

For consequential automated decisions, models that are interpretable by construction are preferable to black boxes explained after the fact [22]. Kolmogorov–Arnold Networks (KANs) [23] place learnable univariate functions on edges and can be converted into symbolic formulas [24], which makes them a candidate for a controller whose decision rule can be read. Symbolic models have been used to explain hyperparameter optimisation after the fact [25] and to distil learned optimisers [26], and KANs have produced interpretable reinforcement-learning policies [27]. A controlled comparison finds that multilayer perceptrons generally match or

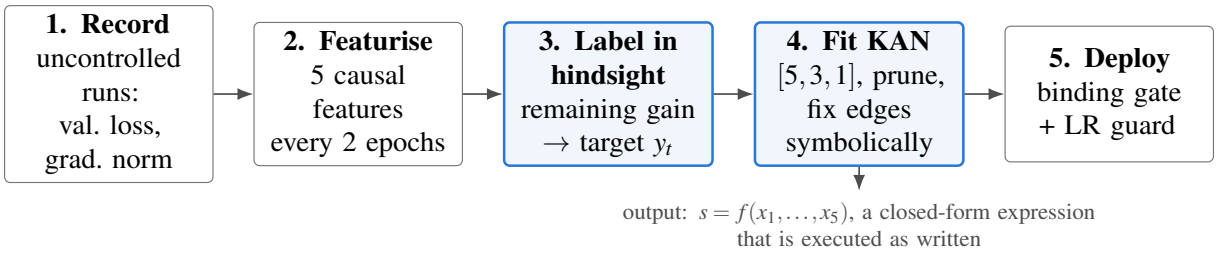


Figure 1: The five steps of the method. Steps 3 and 4 (highlighted) are the customisation of the original KAN method; steps 1, 2 and 5 adapt standard early-stopping practice.

exceed KANs except on symbolic-formula tasks [28]; we therefore claim no accuracy advantage for the KAN and compare it with linear, tree and MLP gates trained on identical labels. Placing a learned symbolic rule between an agent and the actions it may take also resembles shielding in safe reinforcement learning [29].

The method described here customises the original KAN method in two ways. First, it supplies a supervision signal for a control decision that has no ground truth at decision time: hindsight labels derived from completed runs (Step 3). Second, it turns the trained KAN into an executable closed-form decision rule and checks that the rule and the network agree (Step 4). It also adds a deployment guard against a failure mode we observed in closed-loop control (Step 5). The method was developed inside an agentic controller for Kubernetes training jobs, `KAN-KubeAgent`, but it does not depend on Kubernetes, on an LLM or on any particular training framework: it needs only per-epoch validation loss and gradient norm.

What was validated, and how. We collected 144 complete, uncontrolled training runs (5,760 epochs) on four real workloads that differ in dataset, architecture and optimiser, and evaluated the gate against eleven alternatives with two protocols: cross-validation within each workload, grouped by seed, and leave-one-workload-out transfer, in which the gate controls a workload it has never seen. Uncertainty is reported with run-level bootstrap confidence intervals [30] and differences are tested with paired Wilcoxon signed-rank tests [31] with Holm correction [32]. We also tuned a patience rule under the same transfer protocol. We then let the gate control live training processes, including real learning-rate changes, on all four workloads, and compared it on the same processes with standard practice, a reduce-on-plateau learning-rate schedule with patience-based early stopping. No synthetic or simulated data enters any result.

Method details

Overview

The method produces a *gate*: a small function that reads five causal summary statistics of a running training job and returns a stop score $s \in [0, 100]$. The score is mapped to one of three actions: continue ($s < 40$), halve the learning rate ($40 \leq s < 75$) or stop the job ($s \geq 75$). The gate is a Kolmogorov–Arnold Network (KAN) [23] whose edges are replaced, after training, by closed-form univariate functions, so the gate *is* an algebraic expression that an operator can read, recompute and audit. It is trained by *hindsight supervision*: every decision point of a completed training run is labelled with the improvement in validation loss that was actually still to come. Fig. 1 shows the five steps; each is specified below, followed by a step-by-step procedure and the exact commands that reproduce every number in this article.

Step 1: record uncontrolled learning curves

Train a set of runs of the target kind of job to their full epoch budget T with no intervention, and record per epoch t the validation loss v_t, the mean global ℓ_2 gradient norm g_t over the epoch's mini-batches, the validation accuracy and the wall-clock time. The validation loss must come from data the model never trains on. Vary the settings that change the shape of a curve (learning rate, batch size, random seed), because the gate learns from the variety of curve shapes, not from any single run. In this article $T = 40$ and each workload contributes 36 runs (six learning rates $\times$ two batch sizes $\times$ three seeds; Table 1).

Step 2: compute five causal features

At decision epoch t the gate may use only $v_{1:t}$ and $g_{1:t}$. With a window of $w = 5$ epochs, let $W_t = (v_{t-w+1}, \ldots, v_t)$ and Δ_i the successive differences inside W_t. The five features, each in $[0,1]$, are

$$x_1 = \mathrm{clip}\Big(1 - 10\,\frac{\max(v_{t-w+1} - v_t,\, 0)}{|v_{t-w+1}|}\Big) \qquad \text{plateau score} \tag{1}$$

$$x_2 = \mathrm{clip}\Big(\frac{\frac{1}{w}\sum_{s=t-w+1}^{t} g_s}{\max_{s\le t} g_s}\Big) \qquad \text{gradient trend} \tag{2}$$

$$x_3 = \frac{\#\{i : \Delta_i \Delta_{i+1} < 0\}}{w-2} \qquad \text{oscillation (LR-decay benefit)} \tag{3}$$

$$x_4 = 1 - t/T \qquad \text{budget remaining} \tag{4}$$

$$x_5 = \mathrm{clip}\big((t - \arg\min_{s\le t} v_s)/30\big) \qquad \text{epochs since improvement} \tag{5}$$

where $\mathrm{clip}(\cdot)$ clips to $[0,1]$ and the window is truncated to the epochs available when $t < w$. Normalising the gradient norm by the run's own peak (Eq. 2) makes x_2 scale-free across architectures, whose raw gradient norms differ by orders of magnitude. x_3 counts direction changes of the loss, a sign that a smaller learning rate may help. In the software the budget feature is named `gpu_hours_remaining_vs_budget`; in all experiments here it is the fraction of the epoch budget left, not a measured GPU-hour figure. Decisions are taken every two epochs from epoch 4, i.e. at $t \in \{4, 6, \ldots, 38\}$, giving 18 decision points per run.

Step 3: label every decision point in hindsight

Let $m_t = \min_{s\le t} v_s$ be the best validation loss seen by epoch t. Because the recorded run continued to T, the relative improvement that was still to come is known:

$$r_t = \frac{m_t - m_T}{m_t} \;\ge 0. \tag{6}$$

The training target for the decision point is

$$y_t = 100 \cdot \mathrm{clip}\big(1 - r_t/\tau\big), \tag{7}$$

with tolerance $\tau = 5\%$. A point with at least τ of improvement ahead receives $y_t = 0$ (keep training); a point after which the run never improves again receives $y_t = 100$ (stop now). With the decision bands above, a stop ($y_t \ge 75$) means that less than $\tau/4 = 1.25\%$ relative improvement remained. The labels use the future only at training time; the features never do.

This step is the central customisation. The original agentic controller trained its gate to imitate a hand-written three-rule policy, which caps the gate at the quality of that rule, and re-labelled live runs with the gate's own past decisions, which is self-distillation and adds no information. Hindsight labels instead measure what each decision would actually have cost or saved.

Step 4: fit a KAN and fix its edges to symbolic functions

The gate is a KAN of width $[5,3,1]$ with B-spline edges (grid size 5, order $k=3$), implemented with `pykan` 0.2.8. A KAN replaces each weight of a multilayer perceptron with a learnable univariate function on the edge and sums at the nodes [23], following the Kolmogorov–Arnold representation [33]; its output here is

$$s(\mathbf{x}) = \sum_{j=1}^{3} \phi_j^{(2)} \Big(\sum_{i=1}^{5} \phi_{j,i}^{(1)}(x_i) \Big). \tag{8}$$

Fitting regresses s on the targets y_t with mean squared error, using Adam for 200 steps with an ℓ_1 sparsity penalty $\lambda = 0.01$ and entropy penalty 2.0, an 80/20 split of the training decision points for monitoring, pruning of inactive nodes and edges, and a 40-step refit of the pruned network. Then every remaining edge function is replaced by the best-fitting function from the library $\{x, x^2, x^3, \sin\}$ with fitted affine parameters (`auto_symbolic`), and the expression is expanded with SymPy. After this step the network computes the extracted expression, up to numerical precision: the formula shown to an operator is the decision function, not a post-hoc approximation of it. We verify this on every gate by evaluating the network and the formula on the same inputs (*faithfulness*, reported in Method validation).

The library is deliberately narrow. Exponentials, logarithms and nested compositions fit slightly better but produce expressions that are not readable at a glance, which defeats the purpose; the width $[5,3,1]$ is chosen for the same reason.

Step 5: deploy the gate as the binding decision layer

In the released controller (`KAN-KubeAgent`) a LangGraph pipeline observes a running job, computes the features and lets a supervisor agent (a large language model when an API key is configured, otherwise a rule-based fallback) propose an action with a rationale. The proposal is advisory: the gate's band decision is what the executor carries out. *Continue* is a no-op, *adjust learning rate* halves the job's learning rate at the next epoch boundary, and *early stop* terminates the job. The adjust band is not learned as a separate action: by Eq. (7), a score between 40 and 75 corresponds to between 1.25% and 3% relative improvement still ahead, a run that is close to finishing but not finished, and the controller responds by halving the learning rate. Whether a halving helps a given run is not part of the label; it is measured only in the closed-loop experiments (Method validation). Every decision is logged with the features, the score, the formula and an exact additive breakdown of the score into per-feature contributions.

A plateaued job keeps scoring in the adjust band, and an unguarded executor halves its learning rate at every decision point. We therefore add a *learning-rate guard*: an adjustment is executed only if at least two decision points have passed since the previous one, and at most three times per job; a blocked adjustment is logged as deferred and the job continues unchanged. The effect of the guard is measured in closed-loop runs below.

Step-by-step procedure

All commands are run from the root of the released repository.

1. Install Python 3.10 or newer and the dependencies. A GPU is optional; every experiment reported here ran on four CPU cores.

```
pip install -r training/requirements.txt \
            -r kan_gate/requirements.txt \
            -r agents/requirements.txt \
            -r dashboard/backend/requirements.txt
```

2. Record uncontrolled learning curves for each workload (Step 1). Each run is a subprocess of the same training script that the live controller launches.

```
for w in fmnist_cnn mnist_cnn cifar10_cnn fmnist_mlp_sgd; do
  python -m experiments.collect_curves --workload $w \
         --epochs 40 --parallel 2
done
```

3. Evaluate all policies with both protocols, including the sweeps and ablations (Steps 2 to 4 are run inside every fold):

```
python -m experiments.generalization
```

4. Run closed-loop control of live training for each workload; add `--no-guard` to disable the learning-rate guard:

```
python -m experiments.closed_loop --workload cifar10_cnn --gate lowo
```

5. Train the deployable gate on all recorded curves. The learned formula is printed and the checkpoint is saved:

```
python -m kan_gate.train --curves experiments/data/curves*.jsonl
```

6. Start the controller and its dashboard with `./start.sh` on Linux or macOS, or with `.\start.ps1` in Windows PowerShell, and open http://localhost:8000.
7. Regenerate every number, table and figure of this article:

```
python -m experiments.paper_methodsx
```

To apply the method to a new kind of job, record curves of that job and train a gate on them (procedure steps 2 and 5), after checking with step 3 that the gate's trade-off is acceptable; to use a gate without new curves, use one trained on other workloads, which is the situation the leave-one-workload-out protocol below evaluates.

Method validation

Workloads and recorded runs

We recorded 144 complete training runs on four workloads (Table 1). Each workload is a grid of six learning rates, two batch sizes (64, 256) and three seeds (0, 1, 2), trained for $T = 40$ epochs on a fixed 4,000-image subset of the official training split, with validation loss and accuracy measured after every epoch on 2,000 images of the official test split. The CNN workloads use Adam [34]; the MLP workload uses SGD with momentum 0.9 [35], so that the gate also meets an optimiser whose curves look different. The learning-rate grids span stable, fast and unstable training: some runs are still improving at epoch 40, others overfit within a few epochs, and some diverge or stall (Fig. 2). All runs use the same PyTorch [36] training script that the controller launches in deployment, with in-process data loading and two CPU threads per run. The validation images are never used for training; because both the stopping decisions and the reported quality are measured on them, no separate test-set score is reported. On the same software, re-running a configuration with the same seed reproduces its recorded curve exactly: all 84 controlled runs of the closed-loop experiments matched their recordings epoch for epoch up to the first intervention, and a re-run on a fresh installation reproduced the stored losses bit for bit.

Table 1: The four workloads. Each has 36 uncontrolled runs (6 learning rates $\times$ 2 batch sizes $\times$ 3 seeds, 40 epochs). Accuracy is the median over runs of the best validation accuracy; "best epoch" is the median epoch of minimum validation loss; "late" is the share of runs whose minimum falls in the last five epochs; time is the mean wall-clock time per epoch on two CPU threads.

Dataset	Model	Optimiser	Learning rates	Acc. (%)	Best ep.	Late (%)	s/ep.
Fashion-MNIST [37]	CNN, 2 conv	Adam	10^{-4}–0.03	86.9	19.0	38.9	2.35
MNIST [38]	CNN, 2 conv	Adam	10^{-4}–0.03	96.0	14.5	36.1	1.70
CIFAR-10 [39]	CNN, 3 conv	Adam	10^{-4}–0.03	46.3	22.5	38.9	1.36
Fashion-MNIST [37]	MLP 256–128	SGD, mom. 0.9	10^{-3}–0.3	83.5	25.5	27.8	0.43

Evaluation protocol

Counterfactual replay. Every stopping policy is replayed on the recorded curves: the policy sees the run's history up to epoch t and either continues or stops. Learned gates, the median rule and the oracle decide at the decision epochs $t \in \{4, 6, \ldots, 38\}$; patience rules are checked after every epoch, as they are in practice, which gives them a finer stopping granularity. A policy that stops at epoch t is credited with the best validation loss observed up to t, which is what a practitioner who checkpoints the best model obtains. Because a learning-rate change alters the future of a run, *adjust* decisions are treated as *continue* in replay and evaluated in the closed-loop experiments instead.

Metrics. For each run, *compute saved* is $1 - t_{\text{stop}}/T$, averaged over runs. *Regret* is the relative excess of the best validation loss obtained over the best validation loss of the full run, $(m_{t_{\text{stop}}} - m_T)/m_T$; we report its mean, its 90th percentile and its maximum. A stop with regret above 2% is counted as *premature*. We also report the drop in validation accuracy (percentage points) at the best checkpoint.

Policies. The gate (KAN, hindsight labels) is compared with: (i) the same hindsight labels fitted by ridge regression (*linear*), a depth-3 regression tree and a two-layer MLP (32–32) from scikit-learn [40], all using the same features and the same decision bands; (ii) the same KAN trained on the same decision points but labelled by the hand-written three-rule policy of the original controller instead of in hindsight (*rule labels*), which isolates the effect of the label source, and that rule itself; (iii) patience rules with patience 3, 5 and 8 epochs [2]; (iv) the median stopping rule [3], which stops a run whose best loss is worse than the median of the running-average losses of completed runs of the same workload with other seeds at the same epoch; (v) no early stopping; and (vi) a hindsight oracle that stops at the first decision epoch at which less than 1.25% relative improvement remains, an upper reference that no causal policy can reach.

Protocols. Within-workload: three-fold cross-validation inside each workload with folds grouped by seed, so that no test run, and no run sharing a test run's initialisation, is seen in training. *Leave-one-workload-out (LOWO)*: learned gates are trained on all runs of three workloads and tested on every run of the fourth, the situation of a practitioner who deploys a gate on a new kind of job without collecting curves first. Every learned gate is fitted with three seeds (0, 1, 2) and the per-run outcome is averaged over seeds. A learned gate is trained on 432 decision points (24 runs) per fold within a workload and on 1,944 decision points (108 runs) in LOWO.

Statistics. Confidence intervals are 95% percentile intervals from 10,000 bootstrap resamples of runs [30]. Differences between the KAN gate and each alternative are tested per run (144 pairs)

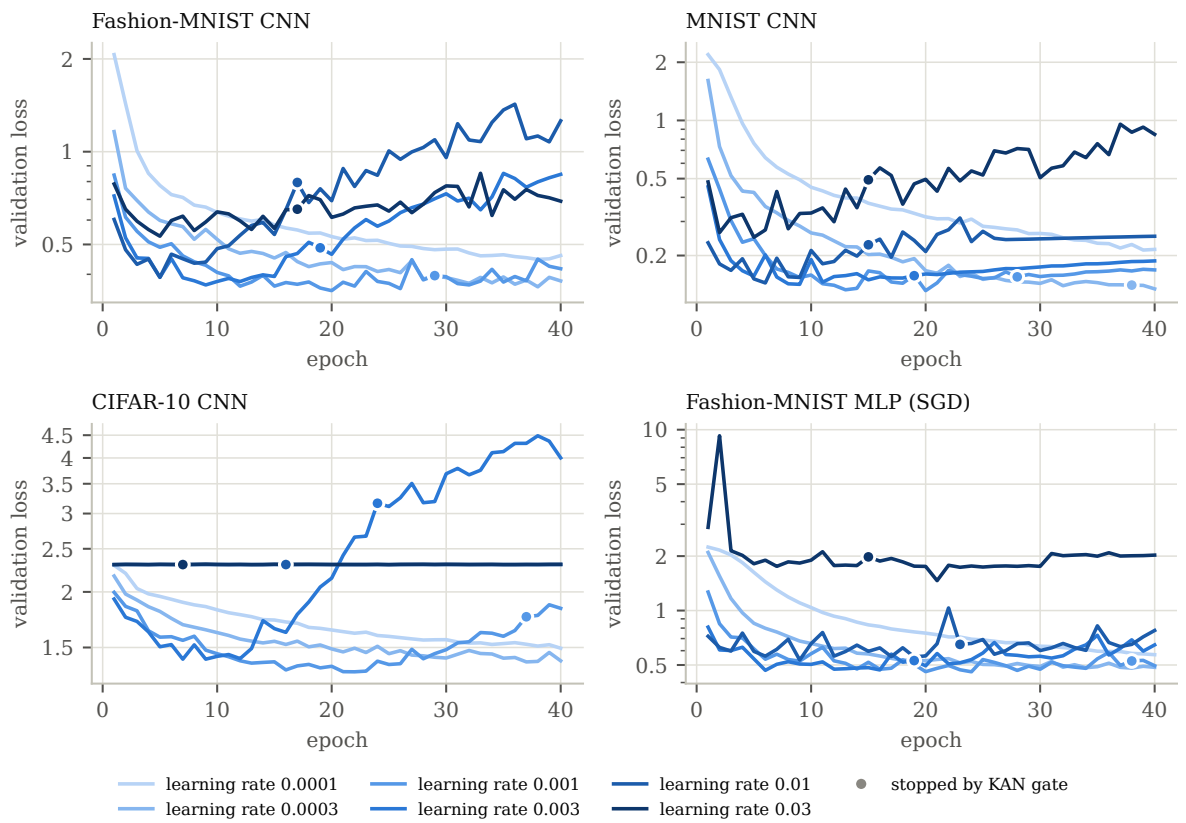


Figure 2: Validation loss of the six learning rates (batch size 64, seed 2) on each workload, log scale. Dots mark where the KAN gate trained on the *other three* workloads stops each run (leave-one-workload-out, mean over three gate seeds); runs without a dot were trained to the full budget.

with the two-sided Wilcoxon signed-rank test [31], zero differences split between ranks, and p-values are Holm-corrected [32] across the nine comparisons of each metric and protocol.

Offline results

Table 2 lists every policy under both protocols, and Fig. 3 places the learned gates and the patience rule on the trade-off between compute saved and regret. Four observations follow.

The gate transfers to unseen workloads. Trained on three workloads and applied to the fourth, the KAN gate saved 25.0% of all epochs (95% CI 20.9 to 29.1) at a mean regret of 0.21% (CI 0.03 to 0.53), with 1.9% of runs stopped prematurely and a mean accuracy loss of 0.11 percentage points. Within a workload it saved 26.9% at 0.58% regret with 3.7% premature stops, so moving to an unseen workload cost no quality and about two percentage points of savings. Transfer works because every feature is normalised per run: a gate learns what a plateau looks like, not the loss level of a particular dataset. Of all learned gates, the KAN had the lowest mean regret and premature-stop rate under transfer (Table 2); the MLP and the depth-3 tree saved more compute but stopped 11.1% and 18.8% of runs prematurely, and their regret was significantly higher ($p < 0.001$ and $p < 0.001$, Table 3).

Hindsight labels are what make the gate useful. The same KAN, trained on the same real decision points but with the labels of the hand-written rule, saved only 15.4% under transfer (17.9% within workloads) and stopped runs significantly later ($p < 0.001$), with no significant difference in regret ($p = 0.134$). The rule itself, applied directly, stopped 22.2% of runs prematurely. Imitating

a rule therefore produces a cautious copy of the rule's thresholds, while hindsight labels produce a gate that stops when the remaining improvement is actually small.

A well-chosen patience rule is as good on the stopping axis. Patience 8 saved 29.3% at 0.22% regret, and the regret of the KAN gate is not significantly different from it ($p = 0.551$); patience stopped slightly earlier ($p < 0.001$). On the frontier of Fig. 3 the patience family and the linear gate lie at or slightly above the KAN gate's curve, i.e. for the stop decision alone they reach the same regret with a few percentage points more compute saved. Shorter patience is clearly worse: patience 3 stopped 23.6% of runs prematurely. The median stopping rule had a mean regret of 55.55%, because in a grid of learning rates it stops slow-starting configurations that later become the best; it is designed for comparing trials, not for deciding when a single job has finished. The value of the KAN gate is therefore not a better stopping trade-off than a tuned patience rule. It is that a single learned rule, with no per-workload tuning, makes calibrated stop decisions on unseen workloads, with a tolerance expressed in units of regret, and is an explicit formula whose every decision can be recomputed and attributed.

A tuned patience rule under the same transfer protocol. Patience 8 was picked with knowledge of all four workloads. To compare like with like, we tuned the patience the way the gate is trained: for each held-out workload, the patience (1 to 20 epochs) that saves the most compute on the other three workloads while keeping their mean regret within a target was applied to the held-out one (Table 4). For targets of 0.20% and 0.30% the tuned rule saved 28.2% and 30.6% at 0.25% and 0.31% mean regret, so on the stop decision alone it is as good as or slightly better than the KAN gate. Its quality, however, transferred less evenly: the patience chosen on the three Adam-trained workloads was too short for the SGD-trained one, whose mean regret reached 1.01%, whereas the KAN gate's mean regret stayed at or below 0.58% on every held-out workload.

Every workload behaves consistently. Table 5 breaks the transfer result down by held-out workload. The KAN gate kept mean regret at or below 0.6% on every held-out workload, including CIFAR-10, the only colour-image workload, and the SGD-trained MLP, the only workload with a different optimiser. Its largest single-run regret, 19.9%, occurs on one unstable SGD run (learning rate 0.3) whose validation loss hovers without progress for about 15 epochs and then reaches its best value in a single isolated dip at epoch 21; every hindsight-trained gate and both patience 5 and patience 8 stop that run before the dip, and only the oracle does not.

Ablations

Table 6 repeats the leave-one-workload-out evaluation with one feature at a time replaced by its training mean, and with three hindsight tolerances. Epochs since improvement (x_5) carries most of the decision: without it the gate stops much earlier and regret rises from 0.21% to 1.72%. The plateau score (x_1) and the oscillation feature (x_3) each add a few percentage points of compute saved at unchanged regret; gradient trend (x_2) and budget remaining (x_4) change little on these workloads. The tolerance τ is a direct and predictable control of the trade-off: $\tau = 2\%$ gives 21.1% saved at 0.11% regret and $\tau = 10\%$ gives 30.2% at 0.59%, so an operator sets the acceptable loss of quality in the units in which it is measured.

The gate is an exact, readable formula

Faithfulness. For each of the 48 KAN gates fitted in the two protocols we evaluated the network and its extracted formula on the same test decision points. The largest absolute difference over all gates and points was 0.12 score points on the 0 to 100 scale, and the median over gates was

Table 2: Offline replay on 144 real runs. Compute saved and mean regret with 95% bootstrap confidence intervals; "prem." is the share of runs stopped with more than 2% regret. Within: seed-grouped 3-fold cross-validation inside each workload. LOWO: trained on three workloads, tested on the fourth. Policies without training (patience, median rule, rule, oracle, no stopping) are identical under both protocols.

	Within-workload			Leave-one-workload-out		
Policy	Saved (%)	Regret (%)	Prem. (%)	Saved (%)	Regret (%)	Prem. (%)
KAN gate, hindsight labels (ours)	26.9 [22.8, 31.0]	0.58 [0.19, 1.09]	3.7	25.0 [20.9, 29.1]	0.21 [0.03, 0.53]	1.9
Linear, hindsight labels	33.3 [28.2, 38.4]	0.42 [0.11, 0.84]	4.2	31.4 [26.2, 36.6]	0.51 [0.20, 0.93]	6.2
MLP, hindsight labels	35.9 [30.8, 40.9]	0.70 [0.34, 1.14]	8.8	37.5 [32.3, 42.7]	0.86 [0.46, 1.32]	11.1
Tree (depth 3), hindsight labels	41.8 [36.5, 46.8]	1.43 [0.97, 1.96]	19.4	41.1 [35.8, 46.3]	1.34 [0.84, 1.92]	18.8
KAN gate, rule labels	17.9 [15.6, 20.3]	0.41 [0.25, 0.61]	6.5	15.4 [13.5, 17.3]	0.27 [0.16, 0.41]	4.9
Hand-written reference rule	33.2 [30.1, 36.0]	1.47 [0.99, 2.00]	22.2	33.2 [30.1, 36.0]	1.47 [0.99, 2.00]	22.2
Patience 3	43.9 [38.7, 49.0]	1.90 [1.32, 2.55]	23.6	43.9 [38.7, 49.0]	1.90 [1.32, 2.55]	23.6
Patience 5	36.0 [30.9, 41.1]	0.55 [0.26, 0.94]	8.3	36.0 [30.9, 41.1]	0.55 [0.26, 0.94]	8.3
Patience 8	29.3 [24.8, 33.8]	0.22 [0.02, 0.56]	2.1	29.3 [24.8, 33.8]	0.22 [0.02, 0.56]	2.1
Median stopping rule	26.9 [20.5, 33.8]	55.55 [35.16, 78.66]	25.7	26.9 [20.5, 33.8]	55.55 [35.16, 78.66]	25.7
Hindsight oracle (not realisable)	43.2 [37.7, 48.5]	0.24 [0.18, 0.31]	0.0	43.2 [37.7, 48.5]	0.24 [0.18, 0.31]	0.0
No early stopping	0.0 [0.0, 0.0]	0.00 [0.00, 0.00]	0.0	0.0 [0.0, 0.0]	0.00 [0.00, 0.00]	0.0

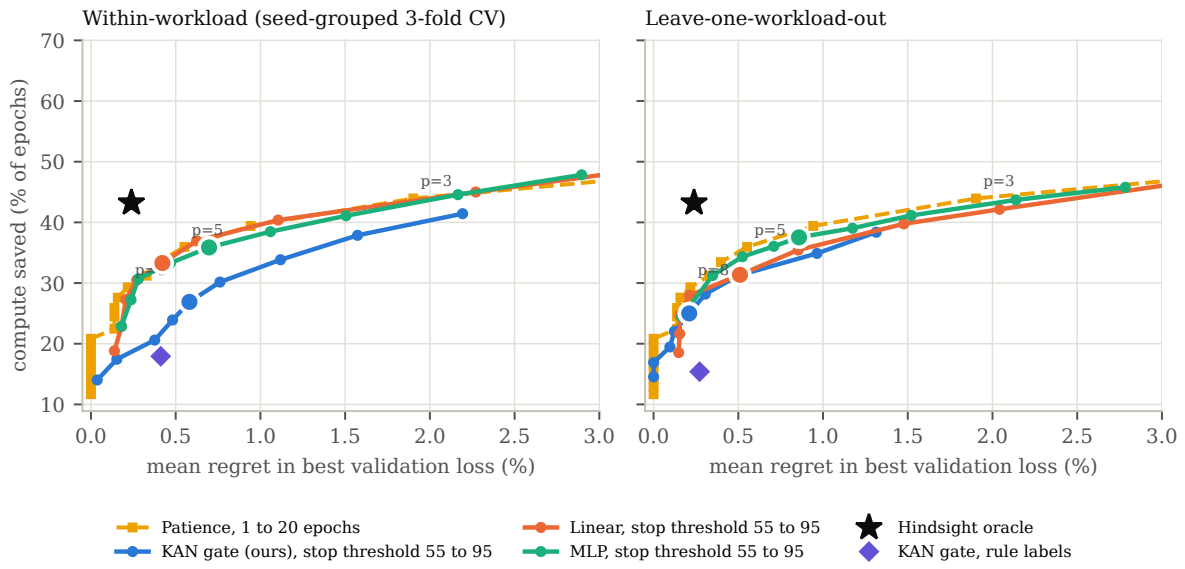


Figure 3: Trade-off between compute saved and mean regret. Learned gates are traced by sweeping the stop threshold from 55 to 95 (large dot: the deployed threshold 75); the dashed line is the patience rule for patience 1 to 20 epochs. Points further up and to the left are better. The median stopping rule (mean regret 55.55%) lies outside the plotted range.

Table 3: Paired comparison of the KAN gate with each alternative over 144 runs: median difference (KAN minus alternative) in stop epoch and in regret (percentage points), with Holm-corrected Wilcoxon signed-rank p-values. A negative stop difference means the KAN gate stops earlier.

	Within-workload				Leave-one-workload-out			
Alternative	Δstop	p	Δregret	p	Δstop	p	Δregret	p
Linear, hindsight labels	1.7	<0.001	0.00	0.676	0.0	<0.001	0.00	0.122
MLP, hindsight labels	2.7	<0.001	0.00	0.042	5.3	<0.001	0.00	<0.001
Tree (depth 3), hindsight labels	5.7	<0.001	0.00	<0.001	6.0	<0.001	0.00	<0.001
KAN gate, rule labels	-1.3	<0.001	0.00	0.163	-1.3	<0.001	0.00	0.134
Hand-written reference rule	0.3	0.001	0.00	0.002	0.7	<0.001	0.00	<0.001
Patience 3	6.2	<0.001	0.00	<0.001	6.7	<0.001	0.00	<0.001
Patience 5	2.5	<0.001	0.00	0.557	3.7	<0.001	0.00	0.122
Patience 8	0.0	0.004	0.00	0.426	0.0	<0.001	0.00	0.551
Median stopping rule	-6.0	0.461	0.00	0.007	-2.7	0.512	0.00	<0.001

Table 4: Patience rule tuned under the leave-one-workload-out protocol. For each held-out workload, the patience is the shortest one (largest saving) whose mean regret on the other three workloads is within the target. Patience is listed per held-out workload in the order Fashion-MNIST CNN, MNIST CNN, CIFAR-10 CNN, Fashion-MNIST MLP (SGD). Worst: largest mean regret of any held-out workload.

Target (%)	Patience chosen	Saved (%)	Regret (%)	Prem. (%)	Worst (%)
0.10	13, 13, 13, 8	23.0	0.16	0.7	0.63
0.20	10, 10, 10, 6	28.2	0.25	2.1	1.01
0.30	8, 8, 8, 6	30.6	0.31	3.5	1.01
0.50	6, 6, 7, 5	33.9	0.45	6.2	1.20

5.3×10^{-4} points; both are far smaller than the width of a decision band (35 points), so the formula and the network can differ in decision only for a score within that distance of a band boundary. The median gate kept 16 active edges after pruning and its expanded formula has 8 additive terms (largest 24).

The price of readability. Fixing the spline edges to symbolic functions is not free. On the held-out 20% of each gate's training decision points, the mean absolute error of the stop score against the hindsight target rose from 14.41 points for the spline network to 23.61 points for the symbolic gate. Because the gate is used only through three wide decision bands, this loss of fit costs little in decisions (Table 2), but users who need a more precise score should keep the spline network and report its symbolic approximation instead, at the cost of exactness.

The deployed gate. Trained on all 144 runs (2,592 decision points), the gate shipped with the controller is

$$s = 21.86x_1 - 19.3x_2 + 20.33x_3 - 14.61x_4 + 72.24x_5 - 3.77\sin(1.49x_3 + 6.51) + 34.99 \quad (9)$$

with $x_1, \ldots, x_5$ as in Eqs. (1)–(5), coefficients rounded to two decimals for display (the controller evaluates the unrounded expression; network and formula agree to 3.2×10^{-4} points on all recorded decision points). Fig. 4 shows each of its 15 active edge functions. An operator can read the gate's behaviour directly from Eq. (9): which features push towards stopping, which towards continuing, and by how much; and for any logged decision the controller reports the score's per-feature contributions, computed from the gate itself.

Table 5: Leave-one-workload-out results per held-out workload: compute saved (%) / mean regret (%).

Policy	Fashion-MNIST CNN	MNIST CNN	CIFAR-10 CNN	F-MNIST MLP (SGD)
KAN gate, hindsight labels (ours)	27.0 / 0.05	31.2 / 0.20	22.9 / 0.01	18.9 / 0.58
Linear, hindsight labels	33.5 / 0.37	41.7 / 0.75	22.8 / 0.09	27.5 / 0.82
MLP, hindsight labels	39.0 / 0.69	45.3 / 1.67	33.1 / 0.11	32.7 / 0.96
Patience 5	36.3 / 0.62	38.4 / 0.16	35.8 / 0.22	33.3 / 1.20
Patience 8	30.6 / 0.24	32.6 / 0.00	29.2 / 0.00	24.7 / 0.63
Hindsight oracle (not realisable)	43.1 / 0.18	46.8 / 0.15	45.0 / 0.41	38.1 / 0.22

Table 6: Feature ablation and tolerance sweep, leave-one-workload-out, KAN gate (three gate seeds). Top: one feature replaced by its training-set mean in training and test. Bottom: hindsight tolerance τ (default 5%).

Feature removed / setting	Saved (%)	Regret (%)	Premature (%)
None (all five features)	25.0	0.21	1.9
x_1 (loss plateau score)	22.8	0.18	1.9
x_2 (gradient trend)	25.2	0.18	1.2
x_3 (lr decay benefit)	22.8	0.18	1.6
x_4 (budget remaining)	25.5	0.19	1.2
x_5 (epochs since improvement)	34.0	1.72	18.8
Tolerance $\tau = 2\%$	21.1	0.11	0.7
Tolerance $\tau = 5\%$	25.0	0.21	1.9
Tolerance $\tau = 10\%$	30.2	0.59	7.4

Cost. Fitting a gate, including pruning and symbolic extraction, took 4.3 s on average on a CPU, and one decision takes about 3.1 ms in pykan, negligible against a training epoch. Evaluating the extracted expression directly, as the static web preview of the controller does in the browser, is cheaper still.

Closed-loop control of live training

Offline replay cannot show what a learning-rate change does. We therefore let the complete controller (feature extraction, supervisor, gate, executor) control live training processes. For each workload we re-ran the 12 configurations of seed 2 under control, with the gate trained on the *other three* workloads, and compared each controlled run with its uncontrolled recording, which has the same seed and settings. Control is asynchronous, as in deployment: the process keeps training while the controller decides, and a learning-rate change takes effect at the next epoch boundary. Until the first intervention, every controlled run reproduced its uncontrolled curve, which confirms that the comparison is paired.

Unseen workloads. Over all 48 runs the controller used 1,502 of 1,920 epochs (21.8% saved) and stopped 22 runs early (Table 7, Fig. 5). The best validation loss was unchanged in the median (0.00%) and not significantly different from uncontrolled training ($p = 0.514$); 13 runs ended more than 2% better and 9 more than 2% worse, and the mean change in validation accuracy at the best checkpoint was -0.19 percentage points. Savings were lower than in offline replay of the same 48 configurations (25.0%) because a run's curve changes after a learning-rate halving, so the controlled runs are no longer the recorded ones, and because a stop decided asynchronously takes effect only at the process's next epoch boundary. The largest loss, 19.9%, is the unstable SGD run discussed above, which the gate stopped before its isolated late minimum.

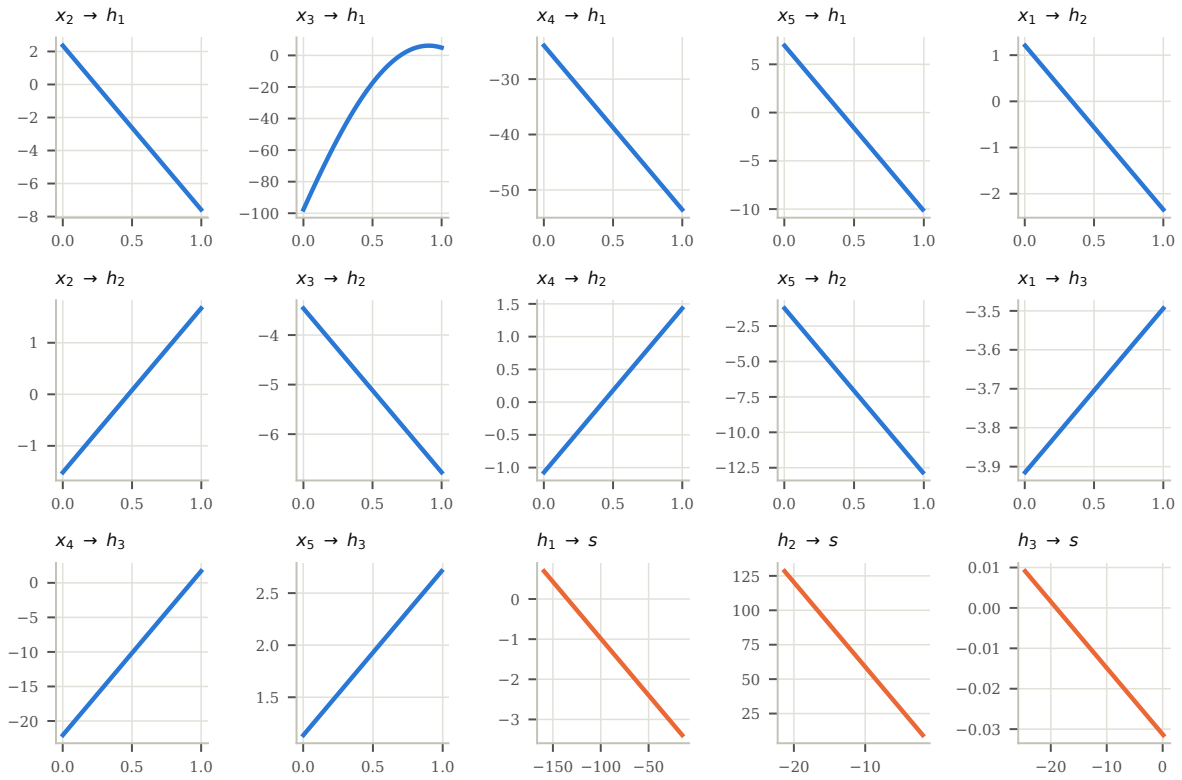


Figure 4: Active edge functions of the deployed gate after symbolic fixing, each drawn over the range of inputs it receives on the recorded decision points. Blue: first layer (feature x_i to hidden node h_j); orange: second layer (hidden node to stop score s).

The learning-rate guard. Without the guard the executor halved the learning rate up to 10 times in one Fashion-MNIST run and up to 12 times in one CIFAR-10 run, leaving those runs at a small fraction of their initial learning rate. With the guard, at most 3 halvings per run were executed (28 further requests were deferred on Fashion-MNIST), and the worst single-run change in best validation loss fell from $+11.1\%$ to $+8.1\%$ on Fashion-MNIST and from $+4.2\%$ to $+2.8\%$ on CIFAR-10, at the cost of fewer epochs saved on Fashion-MNIST (33.3% against 37.1%). Neither condition changed best validation loss significantly. The guard is a deployment safeguard rather than a learned component, and its two parameters (spacing and maximum count) should be revisited for jobs with very different schedules.

Comparison with standard practice in live training

The usual way to automate both decisions in PyTorch or Keras is a reduce-on-plateau learning-rate schedule combined with patience-based early stopping. We ran this policy live on the same 48 configurations: the learning rate is halved (the same step as the controller) when validation loss has not improved by more than a relative 10^{-4} for 3 epochs, and the run stops after 8 epochs without improvement, the best patience of the offline study. Because that patience was chosen with knowledge of all four workloads, the comparison favours the baseline. We also ran a combination that the closed-loop results suggest: the KAN gate trained on the other three workloads decides only when to stop, at the usual decision epochs, and the learning rate follows the same reduce-on-plateau schedule. This variant was specified after the controller's and the baseline's live results were known, so its result is confirmatory only for these 48 configurations. Table 8 reports all three policies, each paired run by run with the uncontrolled recording.

Table 7: Closed-loop control of live training (12 configurations of seed 2 per row). Top: gates trained on the other three workloads, with the learning-rate guard. Bottom: effect of the guard. "Change" is the relative change in best validation loss against the same run uncontrolled (negative is better); "worse" counts runs more than 2% worse; p is the Wilcoxon signed-rank test of controlled against uncontrolled best validation loss; the last column gives learning-rate halvings in total and (in brackets) the most in one run.

Workload / condition	Runs	Epochs used	Saved (%)	Median change (%)	Min / max (%)	Worse	p	LR halvings
Unseen-workload gate, guard on								
Fashion-MNIST CNN	12	388/480	19.2	0.00	−6.1 / 8.5	2	0.469	27 (3)
MNIST CNN	12	318/480	33.8	0.00	−10.1 / 13.3	4	0.969	22 (3)
CIFAR-10 CNN	12	396/480	17.5	0.01	−1.0 / 2.8	2	0.248	28 (3)
Fashion-MNIST MLP (SGD)	12	400/480	16.7	−0.99	−8.1 / 19.9	1	0.148	24 (3)
All four workloads	48	1,502/1,920	21.8	0.00	−10.1 / 19.9	9	0.514	101 (3)
Learning-rate guard off versus on								
Fashion-MNIST CNN, within-workload gate, no guard	12	302/480	37.1	0.00	−7.2 / 11.1	2	0.727	37 (10)
Fashion-MNIST CNN, within-workload gate, guard	12	320/480	33.3	0.00	−4.7 / 8.1	2	1.000	18 (3)
CIFAR-10 CNN, no guard	12	394/480	17.9	0.01	−0.6 / 4.2	3	0.162	79 (12)
CIFAR-10 CNN, guard	12	396/480	17.5	0.01	−1.0 / 2.8	2	0.248	28 (3)

Table 8: Three policies controlling the same 48 live training processes (12 configurations of seed 2 per workload). Epochs: epochs used out of 480 per workload (1,920 in total). Median: median relative change in best validation loss against the same run uncontrolled (%, negative is better). B/W: runs more than 2% better / worse than uncontrolled.

	KAN controller			Plateau LR + patience 8			KAN stop + plateau LR		
Workload	Epochs	Median	B/W	Epochs	Median	B/W	Epochs	Median	B/W
Fashion-MNIST CNN	388	0.00	4/2	352	0.00	1/1	384	0.00	1/1
MNIST CNN	318	0.00	5/4	323	0.00	1/0	322	0.00	1/0
CIFAR-10 CNN	396	0.01	0/2	366	0.00	0/0	396	0.00	0/0
Fashion-MNIST MLP (SGD)	400	−0.99	4/1	412	−2.48	7/0	422	−2.48	7/0
All four workloads	1,502	0.00	13/9	1,453	0.00	9/1	1,524	0.00	9/1

Standard practice is at least as good as the full controller. The reduce-on-plateau baseline used 1,453 of 1,920 epochs (24.3% saved), fewer than the KAN controller's 1,502 (paired Wilcoxon $p = 0.028$), and it improved best validation loss over uncontrolled training ($p = 0.026$; 9 runs more than 2% better, 1 more than 2% worse). The difference in best validation loss between the two controllers was not significant ($p = 0.106$). The controller's weaker point is its learning-rate action: the adjust band responds to runs that are still improving slowly, and on MNIST it halved the learning rate of runs whose learning rate was already too low, which slowed them further (up to 13.3% higher best validation loss on MNIST, against at most 0.5% for the baseline). A plateau schedule halves the learning rate only after the loss has stopped improving.

KAN stop with a plateau schedule. With the learning rate left to the plateau schedule and only the stop decision taken by the KAN gate, the best validation loss was identical to that of standard practice in 48 of the 48 runs: the two rules differed in when they stopped, not in the best model they kept. The gate stopped later, however, using 1,524 epochs (20.6% saved) against 1,453 (paired $p = 0.002$). In live training the gate therefore matches the quality of a well-chosen patience rule but does not save more compute. Its advantages over that rule are of a different kind: the stop decision is an explicit formula learned from outcomes, its tolerance is set in units of regret, and its quality transferred more evenly across held-out workloads (Table 4). If the gate is deployed, we recommend using it as the stop rule together with a standard learning-rate

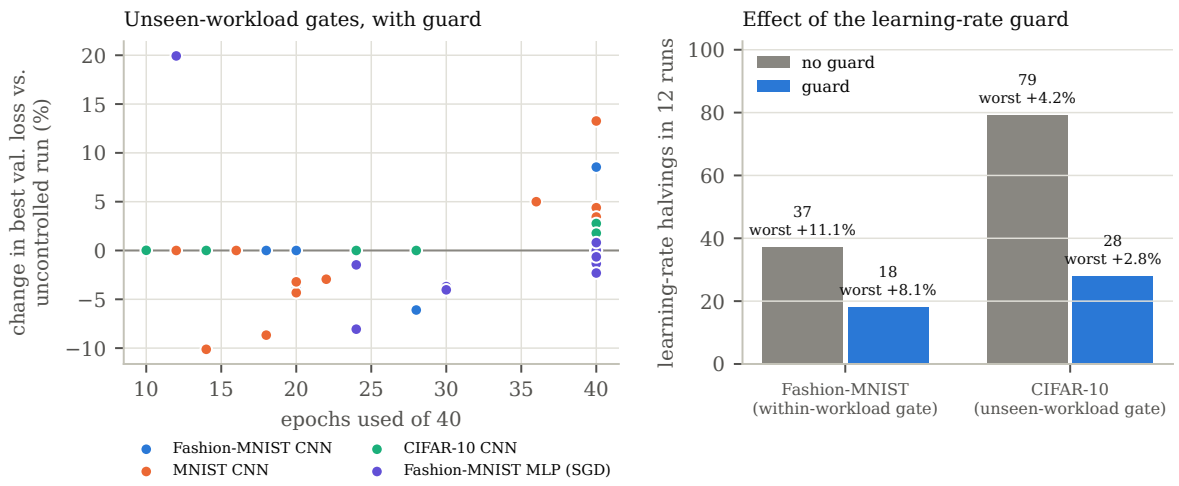


Figure 5: Closed-loop control. Left: for each of the 48 live runs controlled by a gate that had never seen its workload, epochs used and the change in best validation loss against the same run uncontrolled. Right: learning-rate halvings over 12 runs without and with the guard, and the worst single-run change in best validation loss.

schedule, and keeping its adjust band only where it has been validated in closed loop on the target kind of job.

Limitations

- **Scale of the workloads.** All runs are small image classifiers trained on 4,000-image subsets for 40 epochs on a CPU. Large models, long schedules with warm-up and decay, and fine-tuning of pretrained networks produce curve shapes that are not represented here; a gate for such jobs should be retrained on their curves (Steps 1 to 4), and its transfer should be checked with the leave-one-workload-out protocol before it is trusted.
- **No accuracy advantage over simpler rules.** On the same hindsight labels a linear gate reaches a similar trade-off, a patience rule tuned on other workloads is as good on the stop decision alone, and in live training a reduce-on-plateau schedule with patience-based stopping was at least as good as the full controller (Method validation). The contribution of the KAN is an exact, readable and non-linear decision rule, together with the training signal that makes such a rule useful; it is not higher accuracy. Where a linear rule is sufficient for a workload, it is also acceptable and even simpler to audit.
- **The learning-rate action is not learned.** The adjust band is derived from the stop score, not from the measured effect of a learning-rate change. In live control it did not outperform a reduce-on-plateau schedule and harmed some slowly improving runs, so the gate is best used as the stop rule together with a standard learning-rate schedule (Method validation). Learning when a learning-rate change helps would need labels from paired runs with and without the change.
- **Offline replay covers stopping only.** A recorded curve cannot show what a learning-rate change would have done, so the offline benchmark scores the stop decision and treats *adjust* as *continue*. Learning-rate adjustment is evaluated only in the closed-loop experiments, on 12 runs per condition.

- **Hindsight labels need complete runs.** Runs that a gate itself stopped have no observed future and cannot be labelled; harvesting labels from a deployed gate is therefore right-censored. Collecting a share of uncontrolled runs, or runs continued past the gate's stop for a few epochs, avoids this bias.
- **Budget feature.** The budget feature is the fraction of the epoch budget remaining, not measured accelerator time. Deployments with heterogeneous hardware should replace it with a measured quantity and retrain.
- **Supervisor agent not evaluated.** All experiments ran with the rule-based fallback supervisor (no LLM). Because the gate is binding, the supervisor's proposal does not change any action; its rationales were not evaluated.
- **Cluster deployment.** The released controller includes a Kubeflow Trainer client, but all reported runs used the local process client. Behaviour on a live Kubernetes cluster, such as scheduling delays between a decision and its effect, was not measured.

Ethics statements

This work involved no human participants, no animal experiments and no data collected from social media platforms. All datasets used (Fashion-MNIST, MNIST, CIFAR-10) are public benchmark image collections, used under their published terms.

CRediT author statement

Avanish Kasar: Conceptualization, Methodology, Software, Investigation, Formal analysis, Data curation, Visualization, Writing – original draft. **Rupali Pratap Biradar:** Validation, Writing – review & editing. **Jamil Khan:** Investigation, Validation, Writing – review & editing. **Rishikesh R. Nikam:** Supervision, Writing – review & editing.

Data availability

All data and code are openly available at https://github.com/avanishkasar/KAN-KubeAgent: the 144 recorded learning curves (`experiments/data/`), all experiment outputs (`experiments/results/`), the trained gates, and the script `experiments/paper_methodsx.py` that regenerates every number, table and figure in this article.

Acknowledgements

This research did not receive any specific grant from funding agencies in the public, commercial, or not-for-profit sectors.

Declaration of competing interests

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Declaration of generative AI and AI-assisted technologies in the manuscript preparation process

During the preparation of this work the authors used AI-assisted tools to help draft text and software. After using these tools, the authors reviewed and edited the content as needed and take full responsibility for the content of the publication.

References

[1] R. Schwartz, J. Dodge, N. A. Smith, O. Etzioni, Green AI, Communications of the ACM 63 (12) (2020). doi:10.1145/3381831.

[2] L. Prechelt, Early stopping — but when?, in: G. Montavon, G. B. Orr, K.-R. Müller (Eds.), Neural Networks: Tricks of the Trade, 2nd Edition, Springer, 2012, pp. 53–67. doi:10.1007/978-3-642-35289-8_5.

[3] D. Golovin, B. Solnik, S. Moitra, G. Kochanski, J. Karro, D. Sculley, Google Vizier: A service for black-box optimization, in: Proceedings of the 23rd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD), 2017, pp. 1487–1495. doi:10.1145/3097983.3098043.

[4] L. Li, K. Jamieson, G. DeSalvo, A. Rostamizadeh, A. Talwalkar, Hyperband: A novel bandit-based approach to hyperparameter optimization, Journal of Machine Learning Research 18 (185) (2018) 1–52.

[5] L. Li, K. Jamieson, A. Rostamizadeh, E. Gonina, J. Ben-tzur, M. Hardt, B. Recht, A. Talwalkar, A system for massively parallel hyperparameter tuning, in: Proceedings of Machine Learning and Systems (MLSys), Vol. 2, 2020, pp. 230–246.

[6] S. Falkner, A. Klein, F. Hutter, BOHB: Robust and efficient hyperparameter optimization at scale, in: Proceedings of the 35th International Conference on Machine Learning (ICML), 2018, pp. 1437–1446.

[7] T. Akiba, S. Sano, T. Yanase, T. Ohta, M. Koyama, Optuna: A next-generation hyperparameter optimization framework, in: Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD), 2019, pp. 2623–2631. doi:10.1145/3292500.3330701.

[8] R. Liaw, E. Liang, R. Nishihara, P. Moritz, J. E. Gonzalez, I. Stoica, Tune: A research platform for distributed model selection and training, ICML AutoML Workshop; arXiv preprint arXiv:1807.05118 (2018). doi:10.48550/arXiv.1807.05118.

[9] J. George, C. Gao, R. Liu, H. G. Liu, Y. Tang, R. Pydipaty, A. K. Saha, A scalable and cloud-native hyperparameter tuning system, arXiv preprint arXiv:2006.02085 (2020). doi:10.48550/arXiv.2006.02085.

[10] T. Domhan, J. T. Springenberg, F. Hutter, Speeding up automatic hyperparameter optimization of deep neural networks by extrapolation of learning curves, in: Proceedings of the 24th International Joint Conference on Artificial Intelligence (IJCAI), 2015, pp. 3460–3468.

[11] A. Klein, S. Falkner, J. T. Springenberg, F. Hutter, Learning curve prediction with Bayesian neural networks, in: International Conference on Learning Representations (ICLR), 2017.

[12] S. Adriaensen, H. Rakotoarison, S. Müller, F. Hutter, Efficient Bayesian learning curve extrapolation using prior-data fitted networks, in: Advances in Neural Information Processing Systems (NeurIPS), 2023.

[13] M. R. Zhang, N. Desai, J. Bae, J. Lorraine, J. Ba, Using large language models for hyperparameter optimization, NeurIPS Workshop on Foundation Models for Decision Making; arXiv preprint arXiv:2312.04528 (2023). doi:10.48550/arXiv.2312.04528.

[14] S. Liu, C. Gao, Y. Li, Large language model agent for hyper-parameter optimization, arXiv preprint arXiv:2402.01881 (2024). doi:10.48550/arXiv.2402.01881.

[15] Q. Huang, J. Vora, P. Liang, J. Leskovec, MLAgentBench: Evaluating language agents on machine learning experimentation, in: Proceedings of the 41st International Conference on Machine Learning (ICML), 2024, pp. 20271–20309.

[16] J. S. Chan, N. Chowdhury, O. Jaffe, J. Aung, D. Sherburn, E. Mays, G. Starace, K. Liu, L. Maksin, T. Patwardhan, A. Madry, L. Weng, MLE-bench: Evaluating machine learning agents on machine learning engineering, in: International Conference on Learning Representations (ICLR), 2025.

[17] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, Y. Cao, ReAct: Synergizing reasoning and acting in language models, in: International Conference on Learning Representations (ICLR), 2023.

[18] C. Daniel, J. Taylor, S. Nowozin, Learning step size controllers for robust neural network training, in: Proceedings of the AAAI Conference on Artificial Intelligence, Vol. 30, 2016, pp. 1519–1525. doi:10.1609/aaai.v30i1.10187.

[19] Z. Xu, A. M. Dai, J. Kemp, L. Metz, Learning an adaptive learning rate schedule, neurIPS Workshop on Meta-Learning; arXiv preprint arXiv:1909.09712 (2019). doi:10.48550/arXiv.1909.09712.

[20] B. Baker, O. Gupta, R. Raskar, N. Naik, Accelerating neural architecture search using performance prediction, iCLR 2018 Workshop; arXiv preprint arXiv:1705.10823 (2017). doi:10.48550/arXiv.1705.10823.

[21] K. Swersky, J. Snoek, R. P. Adams, Freeze-thaw Bayesian optimization, arXiv preprint arXiv:1406.3896 (2014). doi:10.48550/arXiv.1406.3896.

[22] C. Rudin, Stop explaining black box machine learning models for high stakes decisions and use interpretable models instead, Nature Machine Intelligence 1 (5) (2019) 206–215. doi:10.1038/s42256-019-0048-x.

[23] Z. Liu, Y. Wang, S. Vaidya, F. Ruehle, J. Halverson, M. Soljačić, T. Y. Hou, M. Tegmark, KAN: Kolmogorov-Arnold networks, in: International Conference on Learning Representations (ICLR), 2025, arXiv:2404.19756.

[24] Z. Liu, P. Ma, Y. Wang, W. Matusik, M. Tegmark, KAN 2.0: Kolmogorov-Arnold networks meet science, arXiv preprint arXiv:2408.10205 (2024). doi:10.48550/arXiv.2408.10205.

[25] S. Segel, H. Graf, A. Tornede, B. Bischl, M. Lindauer, Symbolic explanations for hyperparameter optimization, in: Proceedings of the Second International Conference on Automated Machine Learning (AutoML), PMLR 224, 2023.

[26] W. Zheng, T. Chen, T.-K. Hu, Z. Wang, Symbolic learning to optimize: Towards interpretability and scalability, in: International Conference on Learning Representations (ICLR), 2022.

[27] K. Singh, S. Marouani, A. A. Sheikh, P. T. A. Quang, A. Habrard, Interpretable reinforcement learning for load balancing using Kolmogorov-Arnold networks, arXiv preprint arXiv:2505.14459 (2025). doi:10.48550/arXiv.2505.14459.

[28] R. Yu, W. Yu, X. Wang, KAN or MLP: A fairer comparison, arXiv preprint arXiv:2407.16674 (2024). doi:10.48550/arXiv.2407.16674.

[29] M. Alshiekh, R. Bloem, R. Ehlers, B. Könighofer, S. Niekum, U. Topcu, Safe reinforcement learning via shielding, in: Proceedings of the AAAI Conference on Artificial Intelligence, Vol. 32, 2018, pp. 2669–2676. doi:10.1609/aaai.v32i1.11797.

[30] B. Efron, R. J. Tibshirani, An Introduction to the Bootstrap, Chapman & Hall, New York, 1993. doi:10.1201/9780429246593.

[31] F. Wilcoxon, Individual comparisons by ranking methods, Biometrics Bulletin 1 (6) (1945) 80–83. doi:10.2307/3001968.

[32] S. Holm, A simple sequentially rejective multiple test procedure, Scandinavian Journal of Statistics 6 (2) (1979) 65–70.

[33] A. N. Kolmogorov, On the representation of continuous functions of many variables by superposition of continuous functions of one variable and addition, Doklady Akademii Nauk SSSR 114 (1957) 953–956.

[34] D. P. Kingma, J. Ba, Adam: A method for stochastic optimization, in: International Conference on Learning Representations (ICLR), 2015.

[35] I. Sutskever, J. Martens, G. Dahl, G. Hinton, On the importance of initialization and momentum in deep learning, in: Proceedings of the 30th International Conference on Machine Learning (ICML), 2013, pp. 1139–1147.

[36] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga, A. Desmaison, A. Köpf, E. Yang, Z. DeVito, M. Raison, A. Tejani, S. Chilamkurthy, B. Steiner, L. Fang, J. Bai, S. Chintala, PyTorch: An imperative style, high-performance deep learning library, in: Advances in Neural Information Processing Systems (NeurIPS), Vol. 32, 2019, pp. 8024–8035.

[37] H. Xiao, K. Rasul, R. Vollgraf, Fashion-MNIST: A novel image dataset for benchmarking machine learning algorithms, arXiv preprint arXiv:1708.07747 (2017). doi:10.48550/arXiv.1708.07747.

[38] Y. LeCun, L. Bottou, Y. Bengio, P. Haffner, Gradient-based learning applied to document recognition, Proceedings of the IEEE 86 (11) (1998) 2278–2324. doi:10.1109/5.726791.

[39] A. Krizhevsky, Learning multiple layers of features from tiny images, Technical report, University of Toronto (2009).

[40] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, É. Duchesnay, Scikit-learn: Machine learning in Python, Journal of Machine Learning Research 12 (2011) 2825–2830.